Deep Learning for Multi-Class Eye Disease Detection Using Retinal Images

Department of Electrical and Computer Engineering
North South University

***Abstract*—This project explores deep learning techniques for multi-class classification of eye diseases using retinal images. We evaluate state-of-the-art convolutional neural networks (CNNs), transformer-based architectures, and a custom-designed CNN model. The custom CNN, with only 2.10M parameters, achieves 95.07% accuracy, demonstrating efficiency for resource-constrained environments. Model optimization techniques, including pruning and knowledge distillation, further reduce model size while maintaining high accuracy. Explainable AI methods, such as Grad-CAM and LIME, enhance model interpretability, providing insights into decision-making processes. The findings highlight the potential of tailored deep learning models for scalable, accurate eye disease diagnostics.**

***Index Terms*—Eye Disease Detection, Deep Learning, CNN, Transformer, Model Pruning, Knowledge Distillation, Explainable AI**

I. Introduction

Eye diseases affect how your eyes work or look, often causing problems like blurry vision, pain, or even vision loss. They can happen due to aging, genetics, infections, or injuries. Eye diseases such as Cataract, Glaucoma, Diabetic Retinopathy, Macular Scar, and Myopia are leading causes of visual impairment and blindness globally. Early and accurate diagnosis plays a crucial role in timely treatment and prevention of permanent damage. With the rapid advancement in artificial intelligence and computer vision, deep learning models have demonstrated promising potential in automating the detection of ocular diseases from retinal and fundus images. This project focuses on developing and evaluating deep learning-based models for multi-class classification of eye diseases. A comprehensive dataset containing six categories—Macular Scar, Myopia, Cataract, Diabetic Retinopathy, Glaucoma, and Normal—was used, where each image was carefully annotated by medical experts. The project implements multiple approaches, including both state-of-the-art pretrained CNNs and transformer-based architectures. Additionally, a Custom CNN model was designed and trained from scratch to test the feasibility of developing an efficient, lightweight network capable of achieving competitive accuracy while maintaining a small model size. The Custom CNN integrates convolutional layers for feature extraction and fully connected layers for classification, with dropout regularization to prevent overfitting. Despite its significantly smaller parameter count (2.10M), the model achieved a notable accuracy of 93.10%, demonstrating the capability of well-optimized custom architectures for resource-constrained environments.

II. Methodology and Implementation

A. Datasets

In this project, we have evaluated the eye disease detection mechanism using a publicly available Disease dataset. This Dataset includes 1000 Macular Scar images, 1012 Myopia images, 1038 Cataract images, 1098 Diabetic Retinopathy images, 1007 Glaucoma images, and 1074 Normal images. Each image is carefully annotated by experts, ensuring precise and reliable labels for classification tasks. All images in the Dataset are resized to a uniform resolution of 224×224 pixels during preprocessing.

TABLE I: Details of the experimental datasets

Class	Original Images	After Augmentation	Training	Validation	Testing
Macular Scar	1000	2500	1750	250	500
Myopia	1012	2500	1750	250	500
Cataract	1038	2500	1750	250	500
Diabetic Retinopathy	1098	2500	1750	250	500
Glaucoma	1007	2500	1750	250	500
Normal	1074	2500	1750	250	500

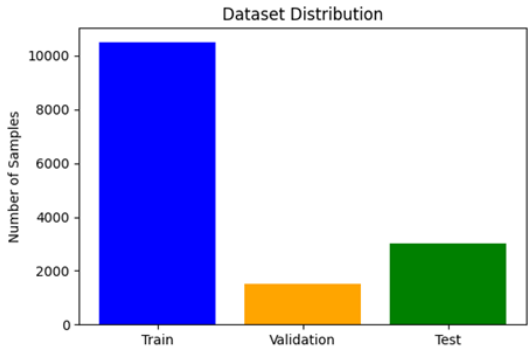


Fig. 1: Split Dataset

B. Data Pre-Processing

Data Pre-Processing ensures preparing the dataset for accurate and reliable disease detection. To avoid overfitting due to an insufficient number of data available in the dataset, an augmentation technique is applied. The code implements a method to increase the dataset size by generating augmentation images, targeting 2500 images per class from the original dataset.

The dataset is split into training, validation, and testing sets at a ratio of 70:10:20. This results in 70% of the images

(10500) used for training, 10% (1500) for validation, and 20% (3000) for testing.

Data augmentation was implemented with specific transformations (listed in Table II) aimed at increasing dataset variability. The specific reasons for using diverse transformations for the augmentation process are as follows:

Flipping balances the dataset by providing mirror images, ensuring the model does not become biased toward any particular edge direction, where edges may appear symmetrically or asymmetrically horizontally. Adjusting brightness introduces variations in illumination levels, enabling the model to detect edges under different lighting conditions. Random helps to distinguish both high-contrast and low-contrast scenarios. Modifying saturation introduces color variations, helping the model learn from different color images, and adjusting hue shifts the color spectrum, allowing the model to detect edges in images with altered or unexpected color palettes.

TABLE II: Data augmentation strategies and parameters

Data Augmentation Strategies	Parameter Values
Original Image	224 × 224 pixels
Random Flip (Horizontal)	$P = 0.5$
Random Brightness	Brightness = 0.5
Random Saturation	Saturation = 0.2
Random Hue	Hue = 0.2

C. Training Phase

In the training phase, the model learns to identify patterns and features from the dataset. In this phase, we implemented techniques like a learning rate scheduler to reduce the learning rate by a factor of 0.1 if the validation loss did not improve for 3 consecutive epochs. Early stopping was implemented with a patience of 5 epochs, halting training if the validation loss did not improve. We consider the following models to evaluate and analyze their performance on the pre-processed data.

TABLE III: Training parameters

Parameter	Values
Optimizer	Adam
Metrics	Accuracy
Loss Function	CrossEntropyLoss
Learning Rate	0.001
Epochs	50
Batch Size	32
Patience	5
Learning Rate Scheduler	Factor = 0.1, Patience = 3

1) Existing Architectures: We initiate the experiment by leveraging established pretrained architectures to ensure a robust foundation for evaluating classification performance on the eye disease detection task. Our selection encompasses 13 prominent CNN-based architectures: MNASNet, VGG16, ResNet50, ResNeXt-50, EfficientNet, EfficientNetV2, MobileNetV2, MobileNetV3, DenseNet121, InceptionV3, ConvNeXt, RegNet, GoogleNet. Each model excels at processing input images and accurately classifying them into one of six disease categories. Their individual performances demonstrate exceptional efficacy, with accuracies ranging from 94.73% to 98.27%.

2) Transformer Learning: The two models are transformer-based architectures for a 6-class image classification task. The first, a Swin Transformer (Swin V2), uses a hierarchical structure with shifted window attention to efficiently process higher-resolution images. The second, a Vision Transformer (ViT), applies global self-attention across image patches for broader context understanding at a slightly lower resolution. Both are pre-trained, and we add custom layers to the classification head to adapt them for the task. Performance is evaluated by calculating the average training and validation loss, along with accuracy on both datasets, reported for each epoch.

3) Custom CNN: The Custom CNN model consists of two parts—one part of the model is used for feature extraction while the other part is used for classification. The feature extraction component involves convolution and pooling layers to effectively capture relevant patterns in the data. On the other hand, the classification component consists of dense layers, also known as fully connected layers, which aid in making accurate predictions based on the extracted features. Moreover, a Dropout layer is used to avoid overfitting of the model.

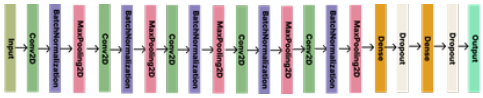


Fig. 2: CNN model architecture

TABLE IV: CNN layer configuration

Layer	Type	Filter Configuration	Output Shape
1	Conv2D	32 filters, 3×3	(32, 224, 224, 32)
2	BatchNormalization	N/A	(32, 224, 224, 32)
3	MaxPooling2D	2×2 pool size	(32, 112, 112, 32)
4	Conv2D	64 filters, 3×3	(32, 112, 112, 64)
5	BatchNormalization	N/A	(32, 112, 112, 64)
6	MaxPooling2D	2×2 pool size	(32, 56, 56, 64)
7	Conv2D	96 filters, 3×3	(32, 56, 56, 96)
8	BatchNormalization	N/A	(32, 56, 56, 96)
9	MaxPooling2D	2×2 pool size	(32, 28, 28, 96)
10	Conv2D	128 filters, 3×3	(32, 28, 28, 128)
11	BatchNormalization	N/A	(32, 28, 28, 128)
12	MaxPooling2D	2×2 pool size	(32, 14, 14, 128)
13	Conv2D	256 filters, 3×3	(32, 14, 14, 256)
14	BatchNormalization	N/A	(32, 14, 14, 256)
15	MaxPooling2D	2×2 pool size	(32, 7, 7, 256)
16	Dense	128 units	(32, 128)
17	Dropout	0.3	(32, 128)
18	Dense	64 units	(32, 64)
19	Dropout	0.3	(32, 64)
20	Output	6 units	(32, 6)

This custom CNN model is remarkably small compared to other existing models. Our custom CNN model contains only 2.10M parameters. The CNN model consists of multiple convolution layers with varying filter sizes, followed by batch normalization and MaxPooling. The network begins with an initial convolutional layer using 32 filters, followed by 64, 96, 128, and 256 filters to capture features. The output is

flattened and passes through a sequence of fully connected layers. Dropout layers are incorporated throughout the dense layers to avoid overfitting. The detailed algorithm is defined in Algorithm 1.

Algorithm 1 Simplified CNN Model

procedure CNNMODEL
  Initialize Inputs:
  Input shape = (224, 224, 3), batch size = 32
  Convolution Layers:
  Apply 2D convolution with 64 filters
  for all filters in [32, 64, 96, 128, 256] **do**
    Conv2D, BatchNormalization, MaxPooling
  end for
  Flatten Layer:
  Flatten the output to shape
  Fully Connected Layers with Dropout:
  for all units, rate in [(128, 0.3), (64, 0.3)] **do**
    Apply Dense layer with units
    Apply Dropout with rate
  end for
  Output Layer:
  Apply Dense layer with num_classes units
end procedure

The performance of the Custom CNN model was evaluated using several key metrics, as described in Section III.

4) Pruning: Pruning is a model compression technique that eliminates redundant or less significant weights from neural networks, reducing their size and computational complexity. In this project, we apply unstructured Magnitude-based pruning to our best-performing CNN model to enhance efficiency while striving to maintain high accuracy. This approach successfully reduces the model size from 7.42 MB to as low as 2.84 MB (at 70% sparsity) while preserving an accuracy of approximately 94.50% after fine-tuning.

The pruning process involves applying `prune.l1_unstructured` to the weights of all convolutional and fully connected layers of the CNN model. We evaluate the model at multiple sparsity levels—20%, 50%, and 70%—to analyze the tradeoff between model size and performance.

Pruning adds two new parameters, `weight_orig` and `weight_mask`, which actually increase the size of the model. Therefore, we use a custom function `permanently_remove_pruning()` to remove pruning-specific parameters that are only necessary during the pruning process.

Additionally, we fine-tune the pruned model for a few epochs (5) using the training and validation datasets to recover any potential accuracy loss, achieving up to 94% accuracy at 70% sparsity.

5) Knowledge Distillation: Knowledge distillation is a technique aimed at transferring the predictive capabilities of a larger, complex model (teacher) to a smaller, more efficient model (student), thereby retaining high performance



(a) Pruning Function (b) Remove Function

Fig. 3: Pruning and Removal Functions

TABLE V: Details of pruning performance

Sparsity Levels (%)	Accuracy w/o Fine-Tune (%)	Accuracy w/ Fine-Tune (%)	Model Size (MB)
20	94.50	95.00	6.33
50	23.90	94.90	4.33
70	17.33	94.50	2.84

while reducing computational demands and model size for deployment on resource-constrained devices. In this project, we implemented knowledge distillation to transfer knowledge from our own custom-designed convolutional neural network (CNN), which is used as the teacher model, to a lightweight student model, enabling efficient deployment on resource-constrained devices.

TABLE VI: Student CNN layer configuration

Layer	Type	Filter Configuration	Output Shape
1	Conv2D	16 filters, 3×3	(32, 224, 224, 16)
2	BatchNormalization	N/A	(32, 224, 224, 16)
3	MaxPooling2D	2×2 pool size	(32, 112, 112, 16)
4	Conv2D	32 filters, 3×3	(32, 112, 112, 32)
5	BatchNormalization	N/A	(32, 112, 112, 32)
6	MaxPooling2D	2×2 pool size	(32, 56, 56, 32)
7	Conv2D	48 filters, 3×3	(32, 56, 56, 48)
8	BatchNormalization	N/A	(32, 56, 56, 48)
9	MaxPooling2D	2×2 pool size	(32, 28, 28, 48)
10	Conv2D	64 filters, 3×3	(32, 28, 28, 64)
11	BatchNormalization	N/A	(32, 28, 28, 64)
12	MaxPooling2D	2×2 pool size	(32, 14, 14, 64)
13	Dense	64 units	(32, 64)
14	Dropout	0.2	(32, 64)
15	Dense	32 units	(32, 32)
16	Dropout	0.2	(32, 32)
17	Output	6 units	(32, 6)

The `train_knowledge_distillation` function facilitates this by training the student model to mimic the teacher's softened outputs (using temperature-scaled softmax) through a KL-divergence loss, combined with a cross-entropy loss for ground-truth labels, weighted by an alpha parameter. This dual-loss approach ensures the student learns both the teacher's nuanced predictions and the true labels. The training process, optimized with Adam and a learning rate scheduler, monitors training and validation losses/accuracies, saving the best model based on validation loss and employing early stopping to prevent overfitting.



```python
# Knowledge Distillation Training Function
def train_knowledge_distillation(custom_model, student_model, train_loader, val_loader, epochs, temperature, alpha_kd, device):
    custom_model.eval()
    student_model.train()
    criterion_ce = nn.CrossEntropyLoss()
    criterion_kl = nn.KLDivLoss(reduction='batchmean')
    optimizer = optim.Adam(student_model.parameters(), lr=LEARNING_RATE)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=3)

    best_val_loss = float('inf')
    epochs_no_improve = 0
    train_losses, val_losses = [], []
    train_accuracies, val_accuracies = [], []

    for epoch in range(epochs):
        student_model.train()
        total_loss, correct, total = 0, 0, 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            with torch.no_grad():
                custom_outputs = custom_model(images)
            student_outputs = student_model(images)
            loss_ce = criterion_ce(student_outputs, labels)
            soft_custom = F.softmax(custom_outputs / temperature, dim=1)
            soft_student = F.log_softmax(student_outputs / temperature, dim=1)
            loss_kl = criterion_kl(soft_student, soft_custom) * (temperature ** 2)
            loss = alpha_kd * loss_kl + (1 - alpha_kd) * loss_ce
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            _, predicted = torch.max(student_outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)

        avg_train_loss = total_loss / len(train_loader)
        train_accuracy = 100 * correct / total
        train_losses.append(avg_train_loss)
        train_accuracies.append(train_accuracy)

        student_model.eval()
        val_loss, val_correct, val_total = 0, 0, 0
        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = student_model(images)
                loss = criterion_ce(outputs, labels)
                val_loss += loss.item()
                _, predicted = torch.max(outputs, 1)
                val_correct += (predicted == labels).sum().item()
                val_total += labels.size(0)

        avg_val_loss = val_loss / len(val_loader)
        val_accuracy = 100 * val_correct / val_total
        val_losses.append(avg_val_loss)
        val_accuracies.append(val_accuracy)

        print(f"Epoch {epoch+1}/{epochs}, Train Loss: {avg_train_loss:.4f}, Train Acc: {train_accuracy:.2f}%, Val Loss: {avg_val_loss:.4f}, Val Acc: {val_accuracy:.2f}%")

        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            epochs_no_improve = 0
            torch.save(student_model.state_dict(), "student_model.pth")
            print("Best student model saved!")
        else:
            epochs_no_improve += 1

        if epochs_no_improve >= PATIENCE:
            print(f"Early stopping triggered at epoch {epoch+1}")
            break

        scheduler.step(avg_val_loss)

    return student_model, train_losses, val_losses, train_accuracies, val_accuracies
```

Fig. 4: Knowledge Distillation Function

The CustomCNN (Teacher) model, with 2.10M parameters and a size of 7.42 MB, achieves a high accuracy of 95.07%, with precision, recall, and F1 scores around 95%. In contrast, the distilled Student model, with only 0.85M parameters and a smaller size of 3.02 MB, maintains a commendable accuracy of 93.50%. The student model provides significant benefits, including a 60% reduction in model size and faster inference, with only a 1.5% drop in performance, making it ideal for resource-constrained environments.

TABLE VII: Details of knowledge distillation performance

Models	Accuracy (%)	#Param. (M)	Model Size (MB)
Custom CNN (Teacher)	95.07	2.10	7.42
Student	93.50	0.85	3.02

6) Explainable AI: Explainable AI (XAI) is a set of techniques that make the decision-making processes of AI models transparent and understandable, moving away from opaque "black-box" systems to foster trust and accountability, particularly in critical domains. In this project, we utilized two XAI methods—Grad-CAM and LIME—to provide visual and interpretable insights into the decision-making process of our teacher model for eye disease classification, enhancing transparency in clinical settings.

a) Grad-CAM: Grad-CAM (Gradient-weighted Class Activation Mapping) generates visual heatmaps that highlight the most influential image regions for the model's classifications by computing gradients from the last convolutional layer, thus providing intuitive insights into the model's focus areas. In our implementation, we developed a `GradCAM_Article` class to generate these heatmaps, enabling visualization of the neural network's predictions for the 6-class eye disease classification task.



```python
class GradCAM_Article:
    def __init__(self, model, target_layer):
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None
        self.target_layer._backward_hooks.clear()
        self.target_layer._forward_hooks.clear()
        self.target_layer.register_forward_hook(self.save_activations)
        self.target_layer.register_full_backward_hook(self.save_gradients)

    def save_activations(self, module, input, output):
        self.activations = output

    def save_gradients(self, module, grad_input, grad_output):
        self.gradients = grad_output[0]

    def generate(self, input_image, target_class=None):
        self.model.eval()
        input_image = input_image.requires_grad_(True)
        output = self.model(input_image)
        if target_class is None:
            target_class = torch.argmax(output, dim=1).item()

        self.model.zero_grad()
        output[0, target_class].backward()
        gradients = self.gradients
        activations = self.activations
        if gradients is None or activations is None:
            raise ValueError("Gradients or activations not captured. Check hook registration.")

        weights = torch.mean(gradients, dim=[2, 3], keepdim=True)
        grad_cam = F.relu(torch.sum(weights * activations, dim=1))

        grad_cam = grad_cam - grad_cam.min()
        grad_cam = grad_cam / (grad_cam.max() + 1e-8)

        grad_cam = F.interpolate(grad_cam.unsqueeze(0), size=(IMAGE_SIZE, IMAGE_SIZE), mode='bilinear', align_corners=False)
        grad_cam = grad_cam.squeeze().detach().cpu().numpy()
        return grad_cam
```

Fig. 5: GradCAM Class

The `__init__` function initializes the `GradCAM_Article` class to generate heatmaps for visualizing a neural network's predictions. It stores the input model and target layer, clears existing hooks on the layer, and sets up placeholders for gradients and activations. It registers forward and backward hooks to capture the target layer's activations and gradients, preparing for GradCAM computation.

The `save_activations` function captures the target layer's feature maps during the forward pass. Triggered by the forward hook, it stores the layer's output in `self.activations`, representing spatial features critical for generating the GradCAM heatmap.

The `save_gradients` function saves the target layer's gradients during the backward pass. Called by the backward hook, it stores the output gradients in `self.gradients`, which indicate the importance of each feature for the model's prediction.

The `generate` function creates a GradCAM heatmap highlighting key image regions for the model's prediction. It runs a forward pass on the input image, selects the target class, and backpropagates to compute gradients. Using stored gradients and activations, it calculates a weighted heatmap, applies ReLU to focus on positive contributions, normalizes it, and resizes it to the image's dimensions, returning the heatmap as a NumPy array.



```python
def overlay_gradcam_on_image(original_image, gradcam, alpha=0.4):
    original_image = original_image.permute(1, 2, 0).detach().cpu().numpy()
    original_image = (original_image - original_image.min()) / (original_image.max() - original_image.min())
    heatmap = cv2.resize(gradcam, (original_image.shape[1], original_image.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    heatmap = heatmap / 255.0
    superimposed_img = heatmap * alpha + original_image * (1 - alpha)
    superimposed_img = np.clip(superimposed_img, 0, 1)
    return superimposed_img
```

Fig. 6: Overlay Function

The `overlay_gradcam_on_image` function blends a GradCAM heatmap with an original image to highlight key prediction regions. It normalizes the image, resizes the heatmap, applies a JET colormap, and combines them using

alpha blending (default 0.4). The output, clipped to [0, 1], visualizes important areas for the model's classification decision.

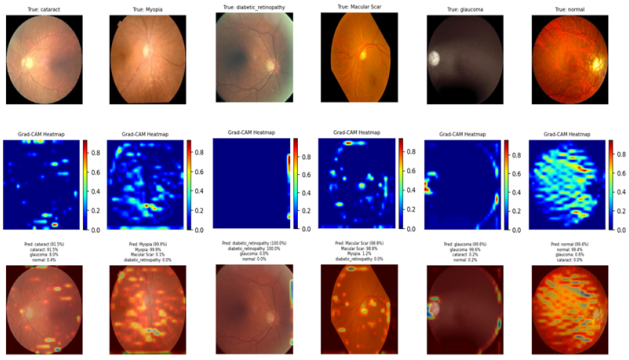


Fig. 7: GradCAM Output Image

The GradCAM output provides a visual representation of the regions in retinal images that the model prioritizes for classifying diseases like Macular Scar or Glaucoma, enabling clinicians to verify whether the model focuses on medically relevant features, such as lesions or optic disc abnormalities.

b) LIME: LIME (Local Interpretable Model-agnostic Explanations) is a technique to explain predictions of complex machine learning models by approximating them locally with simpler, interpretable models. For image classification, LIME perturbs an input image by segmenting it into superpixels (regions) and generates variations by turning these superpixels on or off. It then trains a simpler model (e.g., linear regression) on these perturbed images to identify which superpixels most influence the model's prediction for a specific class, producing an interpretable mask highlighting important regions. We have also used LIME in our project to explain the teacher model's predictions.

The `predict_fn` function prepares a batch of perturbed images for the model by transposing their dimensions to match the expected format (batch, channels, height, width), normalizing pixel values to [0,1], and moving them to the specified device. It then performs a forward pass through the model without gradient computation, applies softmax to obtain class probabilities, and returns these probabilities as a NumPy array for LIME to process.

The `generate` function produces an explanation for a given image. It converts the input PyTorch tensor to a NumPy array, normalizes it to [0,255] for LIME compatibility, and uses `LimeImageExplainer` to generate an explanation. LIME perturbs the image, evaluates predictions via `predict_fn`, and segments the image into superpixels. It identifies the top predicted class (or uses a specified `target_class`) and creates a mask highlighting the top 5 most influential superpixels. The mask is normalized to [0,1] and returned along with the processed image.

The `_segment_image` function segments an input image into superpixels using the SLIC (Simple Linear Iterative

```python
class LIMEExplainer:
    def __init__(self, model, device):
        self.model = model
        self.device = device
        self.explainer = lime_image.LimeImageExplainer()

    def predict_fn(self, images):
        images = np.transpose(images, (0, 3, 1, 2))
        images = torch.from_numpy(images).float() / 255.0
        images = images.contiguous().to(self.device)  #
        with torch.no_grad():
            outputs = self.model(images)
            probs = F.softmax(outputs, dim=1)
        return probs.cpu().numpy()

    def generate(self, image, target_class=None, num_samples=1000, num_segments=50):

        image_np = image.permute(1, 2, 0).cpu().numpy()
        image_np = (image_np - image_np.min()) / (image_np.max() - image_np.min())
        image_np = (image_np * 255).astype(np.uint8)


        explanation = self.explainer.explain_instance(
            image_np,
            self.predict_fn,
            top_labels=1,
            hide_color=0,
            num_samples=num_samples,
            segmentation_fn=lambda x: self._segment_image(x, num_segments)
        )


        if target_class is None:
            target_class = explanation.top_labels[0]


        mask = explanation.get_image_and_mask(
            target_class,
            positive_only=True,
            num_features=5,
            hide_rest=False
        )[1]


        mask = mask.astype(float)
        mask = (mask - mask.min()) / (mask.max() - mask.min() + 1e-8)

        return mask, image_np

    def _segment_image(self, image, num_segments):
        return slic(image, n_segments=num_segments, compactness=10, sigma=1)
```

Fig. 8: LIME Explainer Class

Clustering) algorithm. It takes the image and the desired number of segments (`num_segments`) as inputs and applies SLIC with specified parameters (compactness=10, sigma=1) to group similar pixels into approximately `num_segments` regions, which LIME uses for perturbation and explanation.

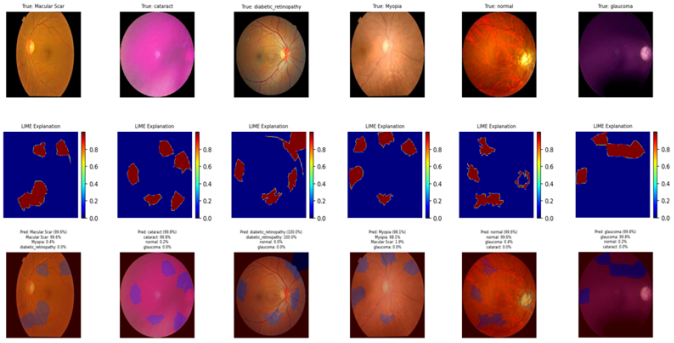


Fig. 9: LIME Output Image

The LIME output highlights specific superpixels in the retinal images that contribute most to the model's classification, such as regions indicating Diabetic Retinopathy or Normal conditions. This method complements Grad-CAM by providing a model-agnostic perspective, allowing clinicians to understand the model's decisions without relying on internal gradients, thus enhancing trust and applicability in medical diagnostics.

By implementing both Grad-CAM and LIME, we ensure that our model's predictions are not only accurate but also

interpretable, addressing the critical need for transparency in AI-driven medical applications. These XAI techniques provide visual evidence of the model's focus, enabling ophthalmologists to validate predictions against clinical knowledge and improving the model's credibility in real-world healthcare settings.

III. Result and Evaluation

The performance of various deep learning models is evaluated using the metrics accuracy, precision, recall, and F1 score. The formulas used for these metrics are as follows:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{1}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2}$$

High precision means a low false positive rate, meaning the model rarely misclassifies negative as positive.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3}$$

High recall means that the model successfully captures most actual positives, reducing missed positive cases.

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

In this project, we explored various deep learning models for the detection of Eye Diseases. Table VIII compares the performance of several deep learning models based on accuracy, precision, recall, F1 score, and program parameters. The custom CNN model achieves the best trade-off with high accuracy (95.07%) while also having the smallest program parameter (2.10M). Other models like ConvNeXt and EfficientNetV2 exhibit higher accuracy and precision but at the cost of significantly larger program parameters.

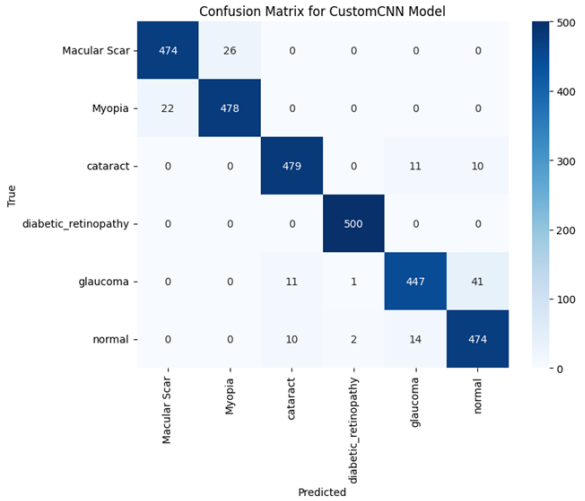


Fig. 10: Confusion Matrix

Figure 10 illustrates the confusion matrix for the custom CNN model, comparing predicted labels with the actual labels.

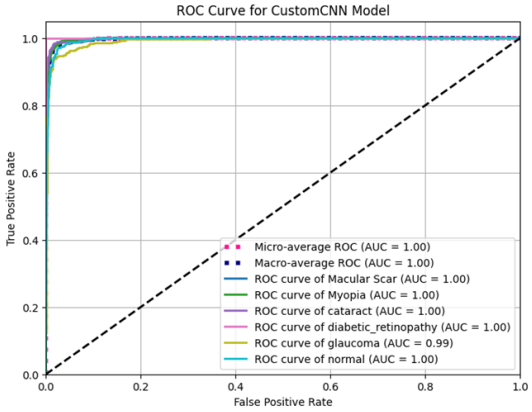


Fig. 11: Receiver Operating Characteristic Curve

Figure 11 illustrates the ROC curve (Receiver Operating Characteristic curve), which plots the true positive rate (sensitivity) against the false positive rate (1 - specificity). It evaluates a model's ability to distinguish between classes, with the area under the curve (AUC) indicating performance; a higher AUC means better discrimination.



Fig. 12: Accuracy and Loss Curve

Figure 12 shows accuracy and loss curves, offering important insights into the model's performance during training and validation. The observed high accuracy and low loss values for both datasets suggest that the model learns efficiently from the training data and generalizes effectively to unseen validation data.

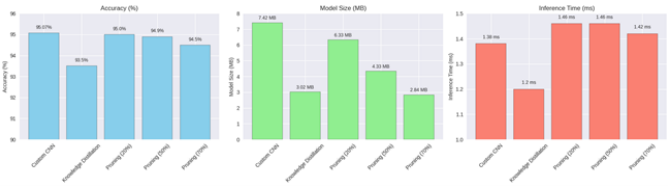


Fig. 13: Comparison Bar Chart

Figure 13 provides a comparative analysis of five models—Custom CNN, Knowledge Distillation, and Pruning at 20%, 50%, and 70% sparsity—across Accuracy, Model Size, and Inference Time.

TABLE VIII: Analysis of performance metrics

Type	Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	#Param. (M)
Convolution	MNASNet	97.77	97.77	97.77	97.76	3.89
	VGG16	96.47	96.47	96.47	96.47	136.50
	ConvNeXt	98.27	98.28	98.27	98.27	88.22
	ResNet50	95.67	95.72	95.67	95.68	24.69
	EfficientNetV2	98.00	98.01	98.00	98.00	20.97
	EfficientNet	97.43	97.45	97.43	97.44	4.80
	RegNet	96.30	96.32	96.30	96.30	18.83
	MobileNetV3	97.80	97.83	97.80	97.80	4.98
	MobileNetV2	96.97	96.97	96.97	96.97	3.01
	InceptionV3	97.47	97.48	97.47	97.47	22.97
	ResNeXt-50	97.50	97.51	97.50	97.50	24.16
	DenseNet121	94.73	94.80	94.73	94.75	6.96
	GoogleNet	94.80	94.88	94.80	94.78	5.61
Transformer	Swin Transformer V2	97.10	97.00	97.00	97.00	27.81
	Vision Transformer	96.10	96.00	96.00	96.00	86.03
OUR	CNN	95.07	95.10	95.07	95.06	2.10

IV. Conclusion

This project presents a comprehensive study on the application of deep learning techniques for multi-class classification of eye diseases using retinal images. Through the integration of various models, including state-of-the-art CNNs, transformer architectures, and a custom-built CNN, we demonstrated robust performance across accuracy, precision, recall, and F1-score metrics. Notably, our Custom CNN model, despite its significantly lower parameter count (2.10M), achieved a commendable accuracy of 95.07%, proving its effectiveness in resource-constrained environments.

Further optimizations through model pruning and knowledge distillation reduced model size and inference time without sacrificing much accuracy, making them highly suitable for real-world deployment. Additionally, the use of Explainable AI techniques such as Grad-CAM and LIME provided meaningful insights into the decision-making process of the models, enhancing transparency and interpretability in clinical settings.

In conclusion, the findings of this project highlight the potential of tailored, efficient deep learning models in the early diagnosis of ocular diseases. With continued refinement and validation, these models can support ophthalmologists in delivering timely, accurate, and scalable diagnostics, particularly in under-resourced healthcare systems.